

ระบบร้านจำหน่ายหนังสืออิเล็กทรอนิกส์ออนไลน์แบบข้ามแพลตฟอร์ม (Folio & Co.)

Cross-Platform Online E-book Store System (Folio & Co.)

จัดทำโดย

นางสาววิพาวัลย์ พลลาภ รหัสนักศึกษา 66332110169-3

เสนอ

อ.ดร.ประภาส ผ่องสนาม

สาขาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

ในปัจจุบัน การอ่านหนังสือและการเข้าถึงเนื้อหาดิจิทัลเปลี่ยนไปจากเดิมอย่างมาก ผู้อ่านนิยมซื้อและอ่านหนังสืออิเล็กทรอนิกส์ผ่านอุปกรณ์ที่หลากหลาย ทั้งคอมพิวเตอร์ สมาร์ตโฟน และแท็บเล็ต เนื่องจากสะดวก พกพาง่าย และได้รับสินค้าทันทีหลังชำระเงิน ขณะเดียวกัน ผู้ประกอบการรายย่อยและนักเขียนอิสระก็ต้องการช่องทางจำหน่ายผลงานของตนเองโดยไม่ต้องพึ่งพาแพลตฟอร์มขนาดใหญ่ทั้งหมด

อย่างไรก็ตาม การสร้างระบบร้านค้าสินค้าดิจิทัลที่สมบูรณ์ไม่ใช่เรื่องง่าย เพราะต้องครอบคลุมหลายส่วน ได้แก่ ระบบสมาชิก การจัดการสินค้า การชำระเงินออนไลน์ การจัดเก็บไฟล์ และที่สำคัญคือการป้องกันไม่ให้ผู้ที่ยังไม่ได้ชำระเงินเข้าถึงไฟล์หนังสือ รวมถึงต้องรองรับผู้ใช้บนหลายแพลตฟอร์ม

ผู้จัดทำจึงพัฒนา "ระบบร้านจำหน่ายหนังสืออิเล็กทรอนิกส์ออนไลน์แบบข้ามแพลตฟอร์ม (Folio & Co.)" โดยใช้เทคโนโลยีสมัยใหม่ ได้แก่ Next.js, Supabase และ Stripe ให้บริการผ่านเว็บแอปพลิเคชัน แอปพลิเคชัน Android และโปรแกรมบน Windows จากโค้ดชุดเดียวกัน เพื่อเป็นต้นแบบของระบบจำหน่ายสินค้าดิจิทัลที่ใช้งานได้จริง ปลอดภัย และขยายต่อได้

1.2 วัตถุประสงค์ของโครงงาน

1. เพื่อพัฒนาระบบร้านจำหน่ายหนังสืออิเล็กทรอนิกส์ออนไลน์ที่รองรับการใช้งานบนเว็บ Android และ Windows
2. เพื่อพัฒนาระบบสมาชิกและการกำหนดสิทธิ์ผู้ใช้ แยกระหว่างลูกค้าและผู้ดูแลระบบ
3. เพื่อพัฒนาระบบชำระเงินออนไลน์ที่ปลอดภัย โดยเชื่อมต่อกับ Stripe และบันทึกสถานะคำสั่งซื้ออัตโนมัติ
4. เพื่อพัฒนาระบบดาวน์โหลดหนังสือที่อนุญาตเฉพาะผู้ที่ชำระเงินแล้ว
5. เพื่อพัฒนาระบบหลังบ้านสำหรับจัดการหนังสือ หมวดหมู่ ไฟล์ และคำสั่งซื้อ
6. เพื่อศึกษาและฝึกปฏิบัติการพัฒนาและเผยแพร่ระบบแบบ Full-Stack บนบริการคลาวด์

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตด้านระบบ

ฝั่งผู้ใช้ทั่วไป

- สมัครสมาชิก เข้าสู่ระบบ และรีเซ็ตรหัสผ่าน
- แสดงรายการหนังสือ ค้นหา และกรองตามหมวดหมู่
- ดูรายละเอียดหนังสือ บทนำ และทดลองอ่านตัวอย่าง
- เพิ่มหนังสือลงตะกร้าและสั่งซื้อ

- ชำระเงินผ่าน Stripe (บัตรเครดิต/เดบิตและ PromptPay)
- ดูประวัติคำสั่งซื้อ และดาวน์โหลดหนังสือจากห้องสมุดของฉันในรูปแบบ PDF หรือ EPUB

ฝั่งผู้ดูแลระบบ

- จัดการข้อมูลหนังสือ (เพิ่ม แก้ไข ลบ เผยแพร่/ซ่อน)
- อัปโหลดไฟล์ปก ไฟล์หนังสือ และไฟล์ตัวอย่าง
- จัดการหมวดหมู่
- ตรวจสอบคำสั่งซื้อทั้งหมด

1.3.2 ขอบเขตด้านแพลตฟอร์มและเทคโนโลยี

- เว็บแอปพลิเคชันแบบ Responsive พัฒนาด้วย Next.js, React และ TypeScript จัดรูปแบบด้วย Tailwind CSS
- แอปพลิเคชัน Android ใช้ Capacitor ห่อเว็บแอปพลิเคชัน
- โปรแกรมบน Windows (.exe) ใช้ Electron ห่อเว็บแอปพลิเคชัน
- ฐานข้อมูล ระบบยืนยันตัวตน และที่เก็บไฟล์ ใช้ Supabase (PostgreSQL)
- ระบบชำระเงินใช้ Stripe Checkout และ Webhook
- เผยแพร่ระบบบน Vercel

1.3.3 ข้อจำกัดของโครงงาน

- หนังสือ ผู้แต่ง บทนำ และเนื้อหาในระบบเป็นข้อมูลสมมติสำหรับทดสอบระบบ ไม่ได้อ้างอิงจากหนังสือหรือบุคคลจริง
- การชำระเงินทำงานในโหมดทดสอบ (Test mode) ของ Stripe ไม่มีการตัดเงินจริง
- แอป Android และ Windows โหลดเนื้อหาจากเว็บบนอินเทอร์เน็ต จึงต้องเชื่อมต่อเครือข่ายขณะใช้งาน
- ตะกร้าสินค้ายังเก็บในอุปกรณ์แต่ละเครื่อง จึงไม่ซิงก์ข้ามแพลตฟอร์ม ส่วนข้อมูลคำสั่งซื้อและห้องสมุดตรงกันทุกแพลตฟอร์ม
- ยังไม่ได้เผยแพร่บนร้านแอปพลิเคชันอย่าง Google Play หรือ Microsoft Store

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้ต้นแบบระบบจำหน่ายหนังสืออิเล็กทรอนิกส์ที่ใช้งานได้จริงบนหลายแพลตฟอร์ม
2. ได้แนวทางการออกแบบความปลอดภัยของระบบสินค้าดิจิทัล เช่น การคำนวณราคาฝั่งเซิร์ฟเวอร์ การตรวจลายเซ็น Webhook การใช้ลิงก์ดาวน์โหลดชั่วคราว และการควบคุมสิทธิ์ระดับแถวข้อมูล (Row Level Security)
3. ได้ประสบการณ์การพัฒนาและนำระบบขึ้นให้บริการจริงด้วยบริการคลาวด์สมัยใหม่

4. สามารถนำโครงสร้างระบบไปประยุกต์ใช้กับสินค้าดิจิทัลประเภทอื่นได้ เช่น เทมเพลต ซอร์สโค้ด หรือคอร์สออนไลน์

บทที่ 2
เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 ระบบพาณิชย์อิเล็กทรอนิกส์และสินค้าดิจิทัล

พาณิชย์อิเล็กทรอนิกส์ (Electronic Commerce: E-Commerce) คือการซื้อขายสินค้าและบริการผ่านเครือข่ายอินเทอร์เน็ต ซึ่งต่างจากการค้าแบบดั้งเดิมตรงที่ไม่จำกัดเวลาและสถานที่ ส่วนประกอบหลักของระบบ E-Commerce ได้แก่ การแสดงสินค้า ตะกร้าสินค้า การชำระเงิน การจัดการคำสั่งซื้อ และการส่งมอบสินค้า

สินค้าดิจิทัล (Digital Product) คือสินค้าที่อยู่ในรูปไฟล์ข้อมูล เช่น หนังสืออิเล็กทรอนิกส์ เทมเพลต ซอร์สโค้ด หรือคอร์สออนไลน์ จุดเด่นคือไม่มีต้นทุนการจัดส่งและส่งมอบได้ทันทีหลังชำระเงิน แต่มีความท้าทายด้านการป้องกันไม่ให้ผู้ที่ไม่ได้ชำระเงินเข้าถึงไฟล์ ระบบนี้จึงออกแบบให้ตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์ก่อนอนุญาตดาวน์โหลดทุกครั้ง

หนังสืออิเล็กทรอนิกส์ (E-book) ในระบบรองรับสองรูปแบบ คือ

- **PDF (Portable Document Format)** เป็นรูปแบบเอกสารที่คงหน้าตาเดิมทุกอุปกรณ์
- **EPUB (Electronic Publication)** เป็นมาตรฐานเปิดสำหรับหนังสืออิเล็กทรอนิกส์ ปรับขนาดตัวอักษรและการจัดหน้าตามหน้าจอได้

2.2 สถาปัตยกรรมเว็บแอปพลิเคชันแบบ Client-Server

ระบบแบ่งเป็นฝั่งผู้ใช้ (Client) ที่แสดงผลและรับคำสั่ง กับฝั่งเซิร์ฟเวอร์ (Server) ที่ประมวลผลและเข้าถึงข้อมูล สื่อสารกันผ่านโปรโตคอล HTTP/HTTPS แบบ Request-Response หลักการสำคัญคือ ไม่เชื่อข้อมูลจากฝั่งผู้ใช้ เช่น ราคาสินค้าและสิทธิ์ ต้องตรวจที่เซิร์ฟเวอร์เสมอ Serverless คือรูปแบบการให้บริการที่ผู้พัฒนาไม่ต้องดูแลเครื่องเซิร์ฟเวอร์เอง ผู้ให้บริการคลาวด์จัดสรรทรัพยากรให้ตามการใช้งาน ระบบนี้ใช้ Vercel รัน API ในรูปแบบดังกล่าว

2.3 เทคโนโลยีฝั่งผู้ใช้และเฟรมเวิร์ก

2.3.1 TypeScript

ภาษาที่พัฒนาต่อยอดจาก JavaScript โดยเพิ่มระบบชนิดข้อมูล (Type System) ช่วยตรวจจับข้อผิดพลาดตั้งแต่ขั้นเขียนโค้ด

2.3.2 React

ไลบรารีสำหรับสร้างส่วนติดต่อผู้ใช้แบบคอมโพเนนต์ (Component-Based) แบ่งหน้าจอเป็นชิ้นส่วนที่นำกลับมาใช้ซ้ำได้ เช่น การ์ดหนังสือและแถบเมนู

2.3.3 Next.js

เฟรมเวิร์กที่สร้างบน React รองรับการเรนเดอร์ฝั่งเซิร์ฟเวอร์ (Server-Side Rendering) การกำหนดเส้นทางตามโครงสร้างโฟลเดอร์ (App Router) และการเขียน API ในโปรเจกต์เดียวกัน ระบบนี้ใช้ Server Components ดึงข้อมูลฝั่งเซิร์ฟเวอร์ และ Route Handlers สร้าง API สำหรับชำระเงิน ดาวน์โหลด และงานของผู้ดูแลระบบ

2.3.4 Tailwind CSS

เฟรมเวิร์ก CSS แบบ Utility-First จัดรูปแบบโดยกำหนดคลาสสำเร็จรูปในองค์ประกอบ HTML ช่วยให้ออกแบบหน้าจอแบบ Responsive ได้รวดเร็วและสม่ำเสมอ

2.4 บริการหลังบ้านแบบ Backend-as-a-Service: Supabase

Backend-as-a-Service (BaaS) คือบริการที่รวมส่วนหลังบ้านสำเร็จรูป เช่น ฐานข้อมูล ระบบยืนยันตัวตน และที่เก็บไฟล์ ช่วยลดภาระการดูแลเซิร์ฟเวอร์ **Supabase** เป็น BaaS แบบโอเพนซอร์สที่สร้างบน PostgreSQL ระบบนี้ใช้ส่วนประกอบดังนี้

1. **PostgreSQL** ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เก็บข้อมูลสินค้า หมวดหมู่ คำสั่งซื้อ และโปรไฟล์ผู้ใช้
2. **Supabase Auth** ระบบสมัครสมาชิก เข้าสู่ระบบ และรีเซ็ตรหัสผ่าน จัดการเซสชันของผู้ใช้
3. **Supabase Storage** ที่เก็บไฟล์แบ่งเป็นถัง (Bucket) แบบสาธารณะสำหรับรูปปก และแบบส่วนตัวสำหรับไฟล์หนังสือ
4. **Row Level Security (RLS)** กลไกควบคุมสิทธิ์ระดับแถวข้อมูลในฐานข้อมูล เช่น ผู้ใช้เห็นเฉพาะคำสั่งซื้อของตนเอง และเฉพาะผู้ดูแลระบบจึงแก้ไขสินค้าได้
5. **Signed URL** ลิงก์เข้าถึงไฟล์แบบชั่วคราวที่มีอายุจำกัด ระบบนี้ออกลิงก์อายุ 60 วินาทีให้เฉพาะผู้ที่ชำระเงินแล้ว

2.5 การควบคุมการเข้าถึงตามบทบาท (Role-Based Access Control)

RBAC คือการกำหนดสิทธิ์ตามบทบาทของผู้ใช้ ระบบนี้แยกบทบาทเป็นลูกค้าและผู้ดูแลระบบ โดยเก็บบทบาทในตารางโปรไฟล์ และตรวจสิทธิ์สามชั้น ได้แก่ การซ่อนเมนูบนหน้าจอ การตรวจฝั่งเซิร์ฟเวอร์ในทุก API และนโยบาย RLS ในฐานข้อมูล เพื่อให้แม้ชั้นใดชั้นหนึ่งมีช่องโหว่ ข้อมูลก็ยังได้รับการป้องกัน

2.6 ระบบชำระเงินออนไลน์: Stripe

Stripe เป็นผู้ให้บริการชำระเงินออนไลน์ ระบบนี้ใช้

- **Stripe Checkout** หน้าชำระเงินสำเร็จรูปที่ Stripe ดูแล ผู้พัฒนาไม่ต้องรับหรือจัดเก็บข้อมูลบัตรเอง ช่วยลดความเสี่ยงด้านความปลอดภัย รองรับบัตรและ PromptPay

- **Webhook** กลไกที่ Stripe ส่งเหตุการณ์กลับมายังเซิร์ฟเวอร์เมื่อชำระเงินสำเร็จหรือล้มเหลว เช่น checkout.session.completed ระบบตรวจลายเซ็น (Signature) ของคำขอก่อนเปลี่ยนสถานะคำสั่งซื้อเป็น "ชำระแล้ว" เพื่อป้องกันการปลอมแปลง
- **Idempotency** การออกแบบให้รับเหตุการณ์ซ้ำได้โดยไม่เกิดผลซ้ำ เนื่องจาก Stripe อาจส่ง Webhook เดิมหลายครั้ง
- **โหมดทดสอบ (Test Mode)** ใช้บัตรทดสอบโดยไม่มีการตัดเงินจริง

2.7 เทคโนโลยีสร้างแอปพลิเคชันข้ามแพลตฟอร์ม

Cross-Platform Development คือแนวทางพัฒนาโค้ดชุดเดียวให้ใช้ได้หลายระบบปฏิบัติการ ระบบนี้ใช้เว็บเป็นแกนหลักและห่อด้วย

- **Capacitor** เครื่องมือห่อเว็บแอปเป็นแอปพลิเคชัน Android โดยเข้าถึงความสามารถของอุปกรณ์ผ่านปลั๊กอิน เช่น การเปิดเบราว์เซอร์สำหรับชำระเงิน
- **Electron** เฟรมเวิร์กสร้างโปรแกรมเดสก์ท็อป (.exe บน Windows) จากเทคโนโลยีเว็บ โดยฝังเอนจินเบราว์เซอร์ Chromium ไว้ในโปรแกรม

ทั้งสองแบบโหลดเนื้อหาจากเว็บที่เผยแพร่บนอินเทอร์เน็ต ทำให้ดูแลโค้ดชุดเดียวและอัปเดตพร้อมกันทุกแพลตฟอร์ม

2.8 การเผยแพร่และเครื่องมือพัฒนา

- **Vercel** แพลตฟอร์มโฮสต์เว็บแบบ Serverless ผูกกับ GitHub ทำให้ทุกครั้งที่ส่งโค้ดใหม่ ระบบ build และเผยแพร่อัตโนมัติ (CI/CD เบื้องต้น)
- **Git และ GitHub** ระบบควบคุมเวอร์ชันและที่เก็บซอร์สโค้ด
- **pdf-lib และ fontkit** ไลบรารีสร้างไฟล์ PDF ภาษาไทยด้วยฟอนต์ที่ฝังในไฟล์ ใช้สร้างหนังสือตัวอย่าง
- **Node.js และ npm** สภาพแวดล้อมรันโปรแกรมและตัวจัดการแพ็กเกจ

บทที่ 3
การออกแบบและพัฒนาระบบ

3.1 ขั้นตอนการดำเนินงาน

โครงงานพัฒนาแบบทีละขั้น (Incremental) โดยทดสอบและนำขึ้นระบบจริงทุกครั้งที่ทำเสร็จหนึ่งส่วน ดังนี้

1. วิเคราะห์ความต้องการและวางแผนระบบ
2. ออกแบบส่วนติดต่อผู้ใช้ (UI) และฐานข้อมูล
3. ตั้งค่าสภาพแวดล้อมพัฒนาและเชื่อมต่อ GitHub กับ Vercel เพื่อเผยแพร่อัตโนมัติ
4. พัฒนาหน้าร้านและเชื่อมต่อฐานข้อมูล
5. พัฒนาระบบสมาชิกและกำหนดสิทธิ์
6. พัฒนาระบบตะกร้า ชำระเงิน และบันทึกคำสั่งซื้อ
7. พัฒนาระบบห้องสมุดและการดาวน์โหลดอย่างปลอดภัย
8. พัฒนาระบบหลังบ้านสำหรับผู้ดูแล
9. จัดทำแอปพลิเคชัน Android และ Windows
10. ทดสอบระบบและสรุปผล

3.2 การวิเคราะห์ความต้องการของระบบ

3.2.1 ผู้ใช้ระบบ

ผู้ใช้	คำอธิบาย
ผู้เยี่ยมชม	ยังไม่ล็อกอิน ดูหนังสือ ค้นหา และทดลองอ่านตัวอย่างได้
ลูกค้า (Customer)	สมาชิกที่ล็อกอินแล้ว ซื้อหนังสือ ดูประวัติ และดาวน์โหลดได้
ผู้ดูแลระบบ (Admin)	จัดการหนังสือ หมวดหมู่ ไฟล์ และดูคำสั่งซื้อทั้งหมด

3.2.2 ความต้องการเชิงหน้าที่ (Functional Requirements)

รหัส	ความต้องการ	ผู้ใช้
F1	สมัครสมาชิก เข้าสู่ระบบ ออกจากระบบ รีเซ็ตรหัสผ่าน	ทุกคน
F2	แสดง ค้นหา และกรองหนังสือตามหมวดหมู่	ทุกคน

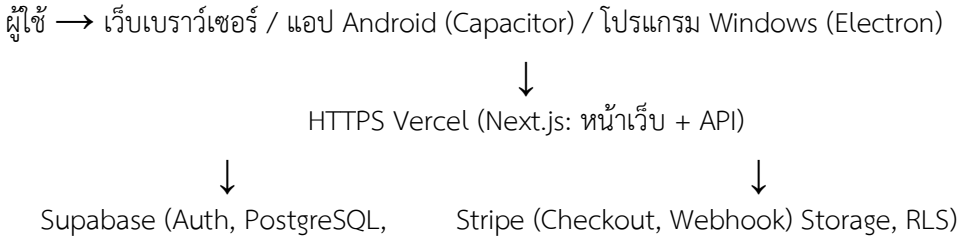
F3	ดูรายละเอียด บทนำ และเปิดไฟล์ตัวอย่าง	ทุกคน
F4	เพิ่มหนังสือลงตะกร้า (เล่มละ 1 ชิ้น)	ลูกค้า
F5	ชำระเงินผ่าน Stripe	ลูกค้า
F6	ดูประวัติคำสั่งซื้อ ชำระต่อ หรือยกเลิกคำสั่งซื้อที่รอชำระ	ลูกค้า
F7	ดาวน์โหลดหนังสือที่ซื้อแล้ว (PDF/EPUB)	ลูกค้า
F8	จัดการหนังสือ อัปโหลดปก ไฟล์หนังสือ ไฟล์ตัวอย่าง	แอดมิน
F9	จัดการหมวดหมู่	แอดมิน
F10	ดูคำสั่งซื้อทั้งหมด	แอดมิน

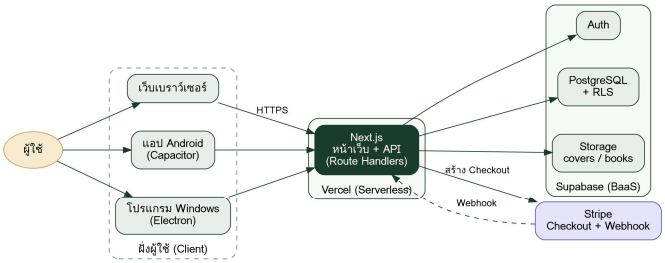
3.2.3 ความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-Functional Requirements)

- **ความปลอดภัย:** ผู้ที่ยังไม่ชำระเงินต้องเข้าถึงไฟล์หนังสือไม่ได้ และผู้ใช้ทั่วไปแก้สิทธิ์หรือสถานะการชำระเงินของตนเองไม่ได้
- **ความถูกต้องของราคา:** ราคาคำนวณจากฐานข้อมูลฝั่งเซิร์ฟเวอร์เท่านั้น
- **การรองรับหลายแพลตฟอร์ม:** ใช้งานได้บนเว็บ Android และ Windows
- **ความพร้อมใช้งาน:** เผยแพร่บนบริการคลาวด์ ไม่ต้องดูแลเซิร์ฟเวอร์เอง
- **การบำรุงรักษา:** โค้ดชุดเดียวสำหรับทุกแพลตฟอร์ม แยกส่วนเป็นคอมโพเนนต์

3.3 สถาปัตยกรรมระบบ

ระบบใช้แนวคิดเว็บเป็นแกนกลาง (Web-Centric) ทุกแพลตฟอร์มเข้าถึงเว็บแอปพลิเคชันเดียวกัน แล้วเรียก API และฐานข้อมูลชุดเดียวกัน



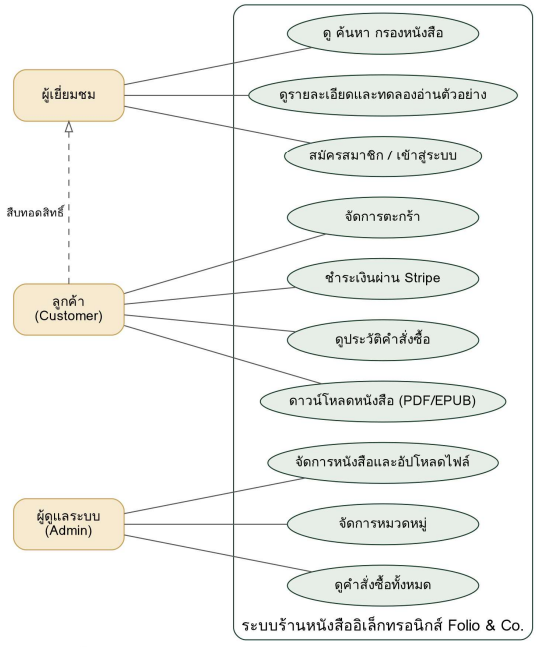


รูปที่ 3.1 สถาปัตยกรรมระบบ Folio & Co.

การแบ่งหน้าที่

ชั้น	ทำหน้าที่	เทคโนโลยี
การแสดงผล	หน้าจอและการโต้ตอบ	React, Tailwind CSS
ตรรกะธุรกิจ	ตรวจสิทธิ์ คำนวณราคา สร้างลิงก์ดาวน์โหลด รับ Webhook	Next.js (Server Components, Route Handlers)
ข้อมูลและไฟล์	จัดเก็บข้อมูลและไฟล์ ควบคุมสิทธิ์ระดับแถว	Supabase
ชำระเงิน	หน้าชำระเงินและแจ้งผล	Stripe

3.4 การออกแบบกรณีการใช้งาน (Use Case)

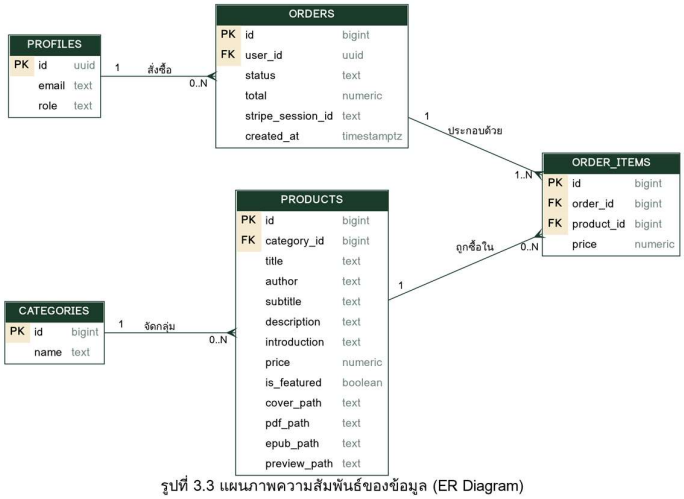


รูปที่ 3.2 แผนภาพกรณีการใช้งาน (Use Case Diagram)

3.5 การออกแบบฐานข้อมูล

ใช้ PostgreSQL บน Supabase มีตารางหลักดังนี้

3.5.1 แผนภาพความสัมพันธ์ (ER Diagram)



รูปที่ 3.3 แผนภาพความสัมพันธ์ของข้อมูล (ER Diagram)

3.5.2 คำอธิบายตาราง

ตาราง	ข้อมูลที่เก็บ	หมายเหตุ
`profiles`	อีเมลและบทบาทของผู้ใช้ เชื่อมกับ Supabase Auth	สร้างอัตโนมัติเมื่อสมัครสมาชิก (Trigger)
`categories`	หมวดหมู่หนังสือ	ลบหมวดแล้วหนังสือกลายเป็นไม่มีหมวด (ON DELETE SET NULL)
`products`	ข้อมูลหนังสือและตำแหน่งไฟล์ใน Storage	เก็บเฉพาะ path ไม่เก็บลิงก์เต็ม
`orders`	คำสั่งซื้อ สถานะ `pending`/`paid`/`failed` ยอดรวม และรหัส Stripe Session	
`order_items`	รายการหนังสือในแต่ละคำสั่งซื้อ พร้อมราคา ณ เวลาที่ซื้อ	กันซื้อเล่มเดิมซ้ำในออเดอร์เดียวด้วย unique index

3.5.3 ที่เก็บไฟล์ (Storage)

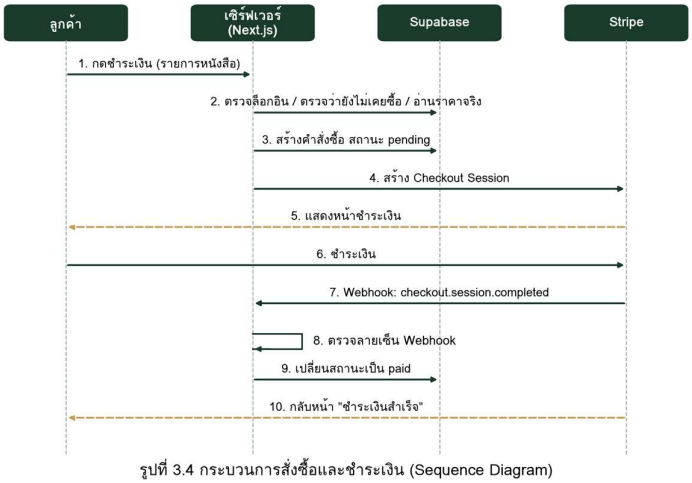
Bucket	การเข้าถึง	เนื้อหา
covers	สาธารณะ	รูปปกหนังสือ
books	ส่วนตัว	PDF EPUB และไฟล์ตัวอย่าง เข้าถึงได้ผ่าน Signed URL เท่านั้น

3.5.4 นโยบายความปลอดภัยระดับแถว (RLS)

ตาราง	ผู้ใช้ทั่วไป	แอดมิน	เซิร์ฟเวอร์ (Service Role)
products, categories	อ่าน	อ่าน/เขียน	ทั้งหมด
orders, order_items	อ่านของตนเอง	อ่านทั้งหมด	สร้าง/แก้ไข (ผ่าน Webhook)
profiles	อ่านของตนเอง	อ่านทั้งหมด	แก้ไขบทบาท

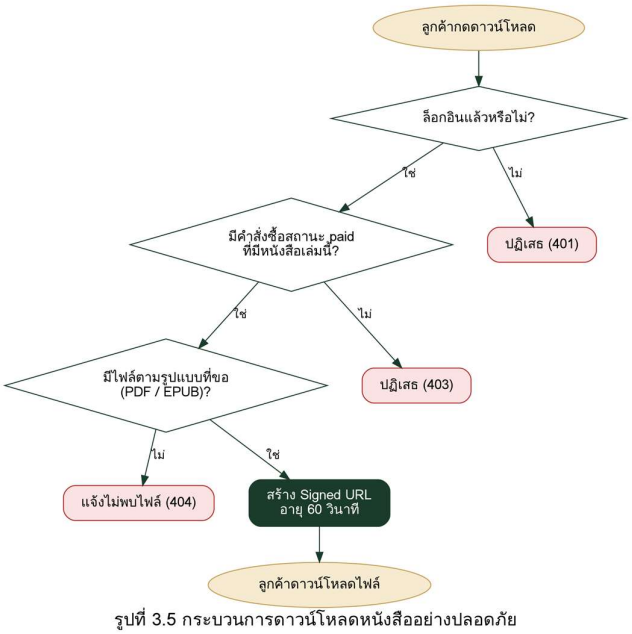
3.6 การออกแบบกระบวนการทำงานหลัก

3.6.1 กระบวนการสั่งซื้อและชำระเงิน



รูปที่ 3.4 กระบวนการสั่งซื้อและชำระเงิน (Sequence Diagram)

3.6.2 กระบวนการดาวน์โหลดหนังสือ



รูปที่ 3.5 กระบวนการดาวน์โหลดหนังสืออย่างปลอดภัย

3.6.3 การดูไฟล์ตัวอย่าง

ไฟล์ตัวอย่าง (บทนำ) ออก Signed URL อายุ 5 นาที โดยไม่ต้องล็อกอิน แต่ใช้เฉพาะ path ไฟล์ตัวอย่าง ไม่เปิดไฟล์เต็ม

3.7 การออกแบบความปลอดภัย

ภัยคุกคาม	**มาตรการป้องกัน**
แก้ราคาจากฝั่งผู้ใช้	อ่านราคาจากฐานข้อมูลฝั่งเซิร์ฟเวอร์เสมอ
ดาวน์โหลดไฟล์โดยไม่จ่ายเงิน	Bucket ส่วนตัว + ตรวจคำสั่งซื้อ `paid` + Signed URL อายุ 60 วินาที
ปลอมการแจ้งชำระเงิน	ตรวจลายเซ็น Webhook และเปลี่ยนสถานะที่ฝั่งเซิร์ฟเวอร์เท่านั้น
ยกตนเองเป็นแอดมิน	ไม่มีนโยบายให้ผู้ใช้แก้ `role` และตั้งแอดมินผ่านฐานข้อมูลเท่านั้น
เข้าถึงหน้า/API แอดมินโดยไม่มีสิทธิ์	ตรวจ `profiles.role` ฝั่งเซิร์ฟเวอร์ทุก API และ RLS ซ้ำอีกชั้น
ข้อมูลบัตรรั่วไหล	ใช้ Stripe Checkout ระบบไม่รับหรือเก็บข้อมูลบัตร

คีย์ลับหลุด	เก็บในตัวแปรสภาพแวดล้อม ไม่ commit ขึ้น GitHub และไม่ใช้ prefix `NEXT_PUBLIC_` กับคีย์ลับ
ซื้อหนังสือเล่มเดิมซ้ำ	ตรวจฝั่งเซิร์ฟเวอร์ และจำกัดเล่มละ 1 ชิ้น

ระบบใช้การป้องกันหลายชั้น (Defense in Depth) คือ ซ่อนปุ่มบนหน้าจอ ตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์ และบังคับด้วย RLS ในฐานข้อมูล

3.8 การออกแบบส่วนติดต่อผู้ใช้

ออกแบบด้วยแนวคิดบรรณาธิการวรรณกรรม (Editorial) ให้ความรู้สึกสงบและหรูหรา

- **สี:** เขียวเข้ม #1B3B2B (สีหลัก), ทอง #C5A059 (เน้น), พื้นหลัง #F6FBF5
- **ฟอนต์:** Playfair Display (หัวข้อ), Plus Jakarta Sans (เนื้อหา), ฟอนต์ไทยสำรอง
- **เลย์เอาต์:** ฝั่งลูกค้าใช้แถบเมนูด้านบนและเน้นภาพปก ฝั่งแอดมินใช้แถบเมนูด้านซ้ายและเน้นตาราง
- **Responsive:** ใช้ Tailwind CSS ปรับตามขนาดหน้าจอ ใช้โค้ดชุดเดียวทั้งเดสก์ท็อปและมือถือ

หน้าจอหลัก: หน้าแรก, รายละเอียดหนังสือ, ตะกร้า, ห้องสมุด, ประวัติคำสั่งซื้อ, เข้าสู่ระบบ, ชำระเงินสำเร็จ, หลังบ้าน (หนังสือ หมวดหมู่ คำสั่งซื้อ)

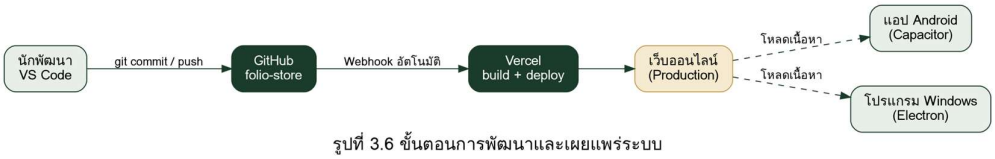
3.9 สภาพแวดล้อมและเครื่องมือที่ใช้พัฒนา

รายการ	รายละเอียด
ระบบปฏิบัติการ	Windows
โปรแกรมเขียนโค้ด	Visual Studio Code
รันไทม์ / ตัวจัดการแพ็กเกจ	Node.js / npm
ควบคุมเวอร์ชัน	Git และ GitHub (`github.com/Wiphawan-Phonlap/folio-store`)
เผยแพร่	Vercel (Deploy อัตโนมัติเมื่อ push)
ทดสอบ Webhook	Stripe CLI
ผู้ช่วยเขียนโค้ด	GitHub Copilot และ Claude
ออกแบบ UI	Stitch

3.10 การนำระบบขึ้นให้บริการ

1. ส่งโค้ดขึ้น GitHub
2. Vercel ตรวจจับและ build อัตโนมัติ (npm run build)

3. ตั้งค่าตัวแปรสภาพแวดล้อมใน Vercel (Supabase และ Stripe)
4. ตั้งค่า Redirect URL ใน Supabase ให้รวมโดเมนจริง และสร้าง Webhook Endpoint ใน Stripe
5. จัดทำแอป Android ด้วย Capacitor และโปรแกรม Windows ด้วย Electron โดยชี้ไปยังเว็บที่เผยแพร่



รูปที่ 3.6 ขั้นตอนการพัฒนาและเผยแพร่ระบบ

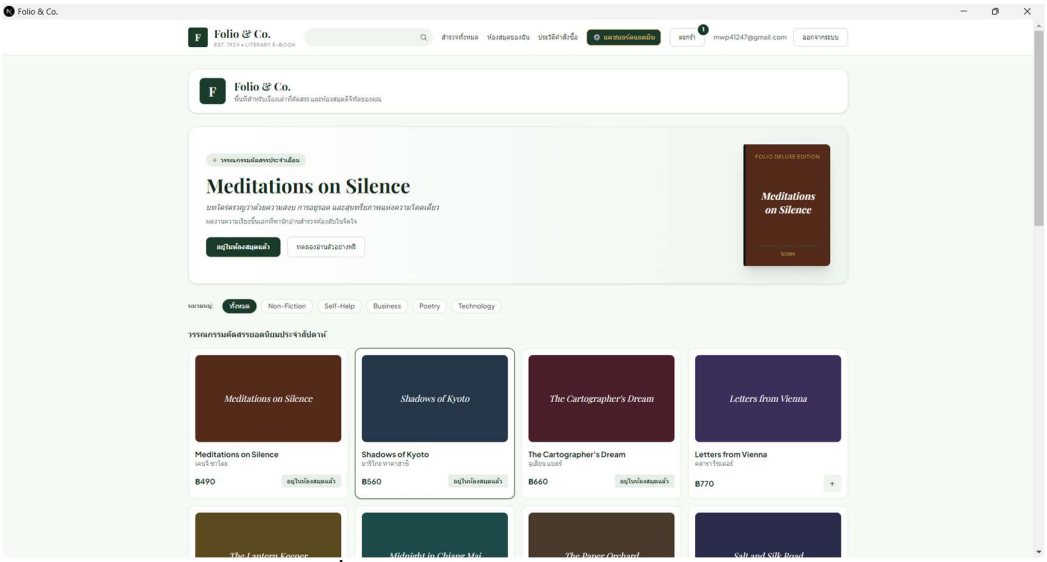
บทที่ 4
ผลการทดลอง

4.1 สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบที่นำมาทดลองคือระบบร้านหนังสืออิเล็กทรอนิกส์ Folio & Co. พัฒนาด้วย Next.js, React, TypeScript, Supabase และ Stripe

รายการ	รายละเอียด
Framework	Next.js 16.4.0
Frontend	React 19.3.0
ภาษาโปรแกรม	TypeScript
ฐานข้อมูล	Supabase
ระบบสมาชิก	Supabase Auth
ระบบจัดเก็บไฟล์	Supabase Storage
ระบบชำระเงิน	Stripe Checkout
Web server	Next.js Production Server
URL ที่ใช้ทดสอบ	http://localhost:3000
รูปแบบการทดสอบ	ทดสอบผ่านเว็บเบราว์เซอร์และเรียก API โดยตรง

4.2 ผลการทดสอบการทำงานของระบบ



รูปที่ 4.1 หน้าแรกของระบบ Folio & Co.

ลำดับ	รายการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบ	สถานะ
1	เปิดหน้าแรกของระบบ	แสดงหน้า Folio & Co. และรายการหนังสือ	แสดงผลสำเร็จ	ผ่าน
2	เปิดหน้าสมัครสมาชิก/เข้าสู่ระบบ	แสดงแบบฟอร์มอีเมลและรหัสผ่าน	แสดงผลสำเร็จ	ผ่าน
3	เปิดหน้าตะกร้าสินค้า	แสดงรายการสินค้า หรือตะกร้าว่าง	แสดงข้อความ “ตะกร้ายังว่างอยู่”	ผ่าน
4	เปิดหน้าห้องสมุด	แสดงหนังสือที่ผู้ใช้ซื้อแล้ว	หน้าเว็บตอบสนองได้	ผ่าน
5	เปิดหน้าประวัติคำสั่งซื้อ	แสดงรายการคำสั่งซื้อของผู้ใช้	หน้าเว็บตอบสนองได้	ผ่าน
6	เปิดหน้ารายละเอียดหนังสือ	แสดงชื่อ ผู้แต่ง ราคา บทนำ และปุ่มซื้อ	แสดงผลสำเร็จ	ผ่าน
7	เปิดหน้า Admin	ระบบแสดงหน้า Admin หรือจัดการสิทธิ์	เส้นทางทำงานและตอบสนองได้	ผ่าน
8	เปิดหน้านโยบายความเป็นส่วนตัวและข้อกำหนด	แสดงข้อมูลเอกสารของระบบ	แสดงผลสำเร็จ	ผ่าน

4.3 ผลการทดสอบการแสดงรายการหนังสือ

จากการทดสอบหน้าแรก ระบบสามารถอ่านข้อมูลหนังสือจากฐานข้อมูลและแสดงผลได้ โดยพบรายการหนังสือจำนวน 25 รายการในหน้าร้าน

ระบบสามารถแสดงข้อมูลสำคัญของหนังสือ ได้แก่

- ชื่อหนังสือ

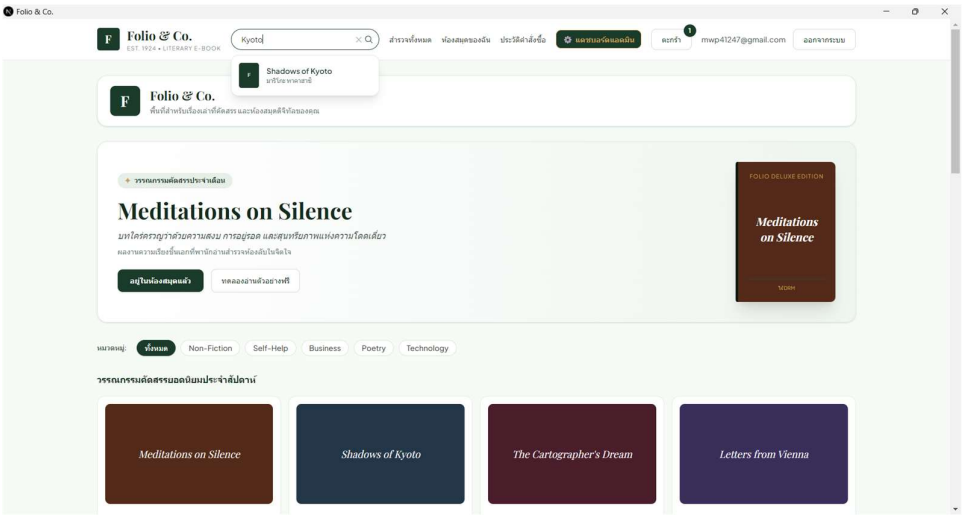
- ผู้แต่ง
- ราคา
- หมวดหมู่
- รูปปก
- ปุ่มเพิ่มลงตะกร้า
- ปุ่มดูรายละเอียด
- หนังสือแนะนำหรือหนังสือเด่น

นอกจากนี้ ระบบยังแสดงหมวดหมู่หนังสือจำนวน 5 หมวดหมู่ ได้แก่

1. Non-Fiction
2. Self-Help
3. Business
4. Poetry
5. Technology

ผลการทดลองพบว่าข้อมูลหนังสือและหมวดหมู่สามารถแสดงผลได้ถูกต้องตามข้อมูลที่จัดเก็บไว้ในฐานข้อมูล

4.4 ผลการทดสอบระบบค้นหา



รูปที่ 4.2 ผลการค้นหาหนังสือด้วยคำว่า “Kyoto”

ทดสอบโดยค้นหาคำว่า Kyoto ผ่านช่องค้นหาหนังสือ

รายการ	ผลการทดสอบ
คำค้น	Kyoto
จำนวนผลลัพธ์	1 รายการ
ผลลัพธ์ที่พบ	Shadows of Kyoto
ผู้แต่ง	มาริโกะ ทาคาฮาชิ
การแสดงผล	แสดงรายการตรงกับคำค้น
สถานะ	ผ่าน

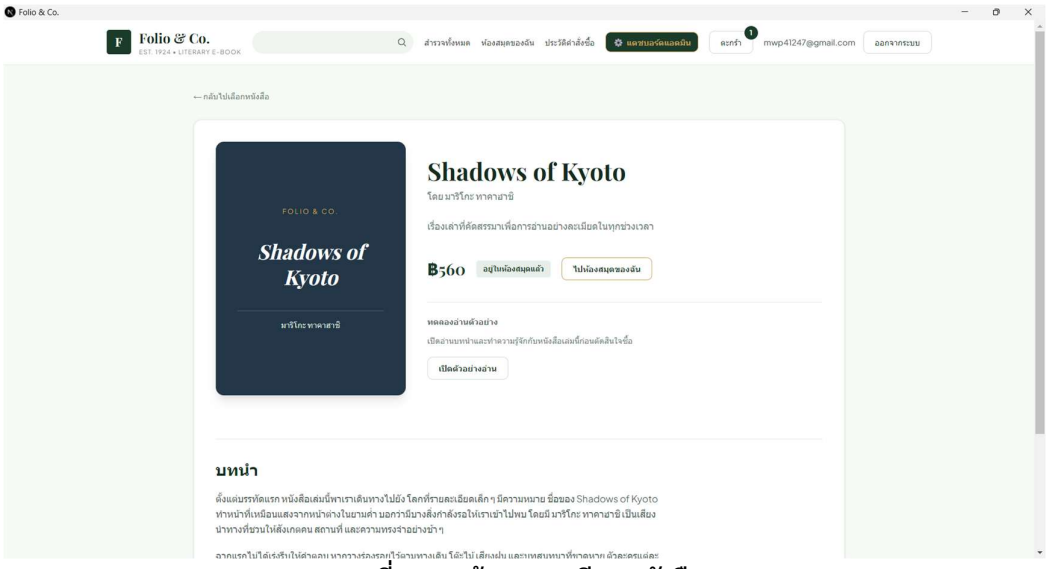
ระบบแสดงข้อความว่า:

ผลการค้นหาสำหรับ “Kyoto” (1 เล่ม)

ผลการทดสอบแสดงให้เห็นว่าระบบสามารถค้นหาหนังสือจากชื่อเรื่องหรือข้อมูลที่เกี่ยวข้องได้อย่างถูกต้อง

นอกจากนี้ API สำหรับการค้นหาหนังสือที่ /api/products/search?q=Kyoto สามารถตอบกลับด้วย HTTP Status Code 200 และส่งข้อมูลในรูปแบบ JSON ได้สำเร็จ

4.5 ผลการทดสอบหน้ารายละเอียดหนังสือและตัวอย่างอ่าน



รูปที่ 4.3 หน้ารายละเอียดหนังสือ

ทดสอบหน้ารายละเอียดของหนังสือ Shadows of Kyoto ที่ URL /books/2

ระบบสามารถแสดงข้อมูลต่อไปนี้:

- ชื่อหนังสือ
- ชื่อผู้แต่ง

- ราคา 560 บาท
- คำอธิบายหนังสือ
- บทนำ
- ปุ่มเพิ่มลงตะกร้า
- ปุ่มซื้อทันที
- ปุ่มเปิดตัวอย่างอ่าน

API ตัวอย่างอ่านที่ /api/books/2/preview ตอบกลับด้วย HTTP Status Code 200 และส่งข้อมูลในรูปแบบ JSON ได้สำเร็จ

จึงสรุปได้ว่าระบบสามารถให้ผู้ใช้ตรวจสอบรายละเอียดและอ่านตัวอย่างหนังสือก่อนตัดสินใจซื้อได้

4.6 ผลการทดสอบระบบตะกร้าสินค้า

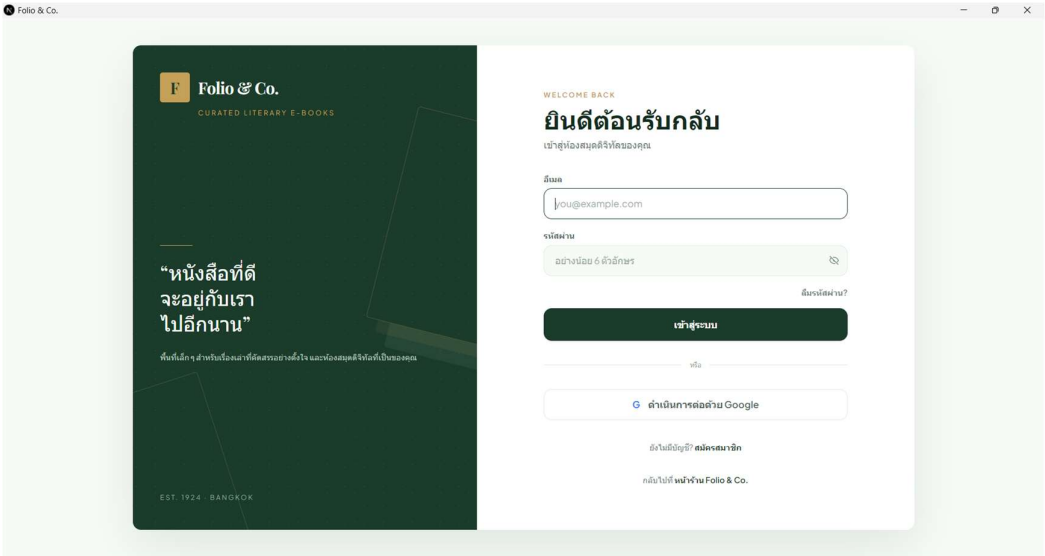
เมื่อเปิดหน้าตะกร้าสินค้าในกรณีที่ยังไม่มีสินค้า ระบบแสดงผลดังนี้:

- แสดงหัวข้อ “ตะกร้าสินค้า”
- แสดงข้อความ “ตะกร้ายังว่างอยู่”
- แสดงคำแนะนำให้ผู้ใช้เลือกหนังสือ
- แสดงปุ่มกลับไปเลือกหนังสือ

ผลการทดลองเป็นไปตามที่ออกแบบไว้ ระบบสามารถจัดการกรณีที่ไม่มีสินค้าในตะกร้าได้โดยไม่เกิดข้อผิดพลาด

4.7 ผลการทดสอบการตรวจสอบสิทธิ์

ได้ทดสอบเรียก API สำคัญโดยยังไม่ได้เข้าสู่ระบบ



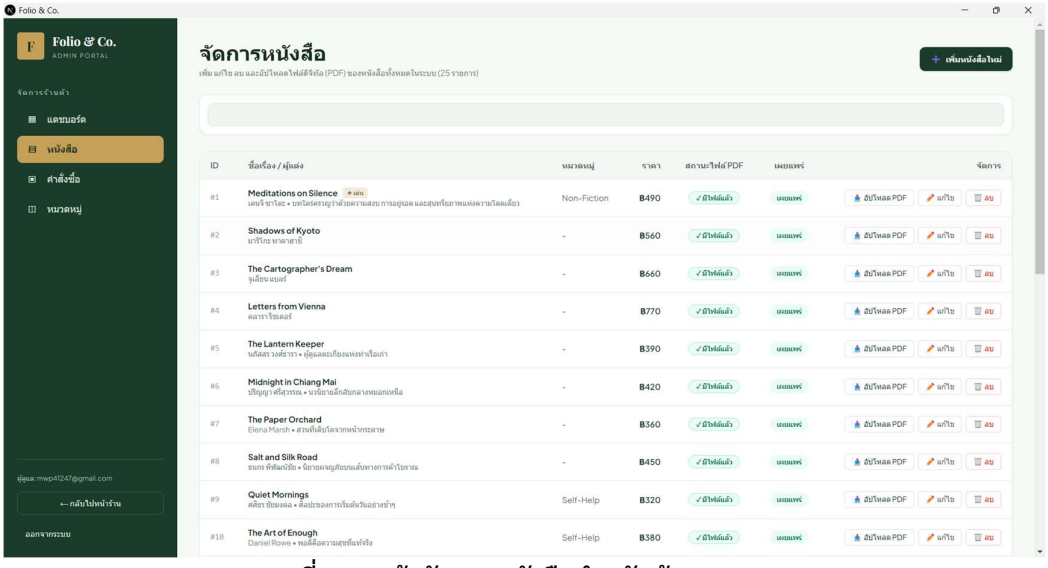
รูปที่ 4.6 หน้าเข้าสู่ระบบ

ตารางที่ 4.2 ผลการทดสอบการตรวจสอบสิทธิ์

API ที่ทดสอบ	กรณีทดสอบ	ผลลัพธ์	สถานะ
/api/checkout	ไม่ส่งหมายเลขคำสั่งซื้อ	HTTP 400 พร้อมข้อความแจ้งให้ระบุ orderId	ผ่าน
/api/checkout	ส่ง orderId แต่ไม่เข้าสู่ระบบ	HTTP 401 แจ้งให้เข้าสู่ระบบก่อน	ผ่าน
/api/library/download	ส่งพารามิเตอร์ไม่ถูกต้อง	HTTP 400 แจ้งว่าพารามิเตอร์ไม่ถูกต้อง	ผ่าน
/api/library/download	ขอไฟล์โดยไม่เข้าสู่ระบบ	HTTP 401 แจ้งให้เข้าสู่ระบบ	ผ่าน
/api/library/owned	เรียกโดยยังไม่มีหนังสือที่ซื้อ	ส่งรายการว่างกลับมา	ผ่าน

ผลการทดลองแสดงให้เห็นว่า API มีการตรวจสอบข้อมูลนำเข้าและสถานะการเข้าสู่ระบบก่อนดำเนินการกับข้อมูลสำคัญ

4.8 ผลการตรวจสอบการ Build และคุณภาพโค้ด



รูปที่ 4.8 หน้าจัดการหนังสือสำหรับผู้ดูแลระบบ

ได้ดำเนินการตรวจสอบระบบด้วยคำสั่งดังต่อไปนี้:

npm run build

ผลการทดสอบ:

- ไม่พบ Error
- พบ Warning จำนวน 16 รายการ
- Warning ส่วนใหญ่เกิดจากไฟล์ JavaScript ที่ Capacitor/Android สร้างขึ้นอัตโนมัติ
- Warning ไม่ได้ทำให้ระบบ Build ล้มเหลว

ดังนั้นระบบสามารถ Build และเริ่มต้นใช้งานใน Production Mode ได้สำเร็จ

4.9 สรุปผลการทดลองบทที่ 4

จากการทดลองพบว่า ระบบ Folio & Co. สามารถทำงานได้ตามวัตถุประสงค์หลัก ได้แก่

1. แสดงรายการหนังสือจากฐานข้อมูลได้
2. ค้นหาหนังสือได้
3. กรองหนังสือตามหมวดหมู่ได้
4. แสดงรายละเอียดและบทนำของหนังสือได้
5. จัดการตะกร้าสินค้าได้
6. ตรวจสอบการเข้าสู่ระบบก่อนทำรายการสำคัญได้
7. ป้องกันการดาวน์โหลดไฟล์โดยผู้ไม่มีสิทธิ์ได้
8. รองรับเส้นทางของระบบสมาชิก ห้องสมุด และระบบ Admin
9. ระบบสามารถ Build เป็น Production ได้สำเร็จ
10. API สำคัญสามารถตอบสนองและส่งข้อมูลตามรูปแบบที่กำหนด

โดยผลการทดลองเบื้องต้นพบว่าการทำงานของระบบผ่านเกณฑ์ที่กำหนดในส่วนของการแสดงผล การค้นหา การตรวจสอบสิทธิ์ และการจัดการข้อมูลพื้นฐาน

บทที่ 5
สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการพัฒนาระบบ

ระบบ Folio & Co. เป็นระบบร้านหนังสืออิเล็กทรอนิกส์ที่ออกแบบให้ผู้ใช้สามารถค้นหาและเลือกซื้อหนังสือดิจิทัลผ่านเว็บไซต์ได้ ระบบประกอบด้วยส่วนสำคัญ ได้แก่ ระบบแสดงสินค้า ระบบค้นหา ระบบตะกร้าสินค้า ระบบสมาชิก ระบบชำระเงิน ห้องสมุดดิจิทัล และระบบจัดการสำหรับผู้ดูแลระบบ

จากผลการทดลองพบว่าระบบสามารถทำงานตามขอบเขตที่กำหนดไว้ โดยหน้าเว็บหลักสามารถแสดงข้อมูลหนังสือจริงจากฐานข้อมูล และผู้ใช้สามารถค้นหาหนังสือ รวมถึงเปิดดูรายละเอียดและตัวอย่างอ่านได้

ในด้านความปลอดภัย ระบบมีการตรวจสอบสถานะการเข้าสู่ระบบก่อนดำเนินการซื้อหรือดาวน์โหลดหนังสือ อีกทั้งยังมีการสร้างลิงก์ดาวน์โหลดแบบชั่วคราวสำหรับไฟล์หนังสือ ทำให้ผู้ใช้ไม่สามารถเข้าถึงไฟล์โดยตรงได้หากไม่มีสิทธิ์

5.2 อภิปรายผลการทดลอง

ผลการทดลองแสดงให้เห็นว่า การใช้ Next.js ช่วยให้ระบบสามารถรวมส่วนแสดงผลและ API ไว้ในโครงการเดียวกัน ทำให้การพัฒนาและดูแลระบบทำได้สะดวกขึ้น

การใช้ Supabase ช่วยลดความซับซ้อนในการจัดการฐานข้อมูล การยืนยันตัวตน และการจัดเก็บไฟล์ เนื่องจากมีบริการเหล่านี้รวมอยู่ในแพลตฟอร์มเดียวกัน

การใช้ Stripe Checkout ทำให้ระบบสามารถแยกกระบวนการชำระเงินออกจากระบบหลักได้อย่างเหมาะสม และใช้ Webhook เพื่อตรวจสอบผลการชำระเงินจากฝั่งเซิร์ฟเวอร์ ส่งผลให้สถานะคำสั่งซื้อไม่ขึ้นอยู่กับข้อมูลที่ส่งมาจากฝั่งผู้ใช้เพียงอย่างเดียว

จากการทดสอบ API พบว่าระบบมีการตรวจสอบข้อมูลนำเข้า เช่น หมายเลขคำสั่งซื้อ รูปแบบไฟล์ และสถานะการเข้าสู่ระบบ ซึ่งช่วยลดความเสี่ยงจากการเรียก API โดยไม่ถูกต้อง

อย่างไรก็ตาม การทดลองครั้งนี้เป็นการทดสอบในสภาพแวดล้อมภายในเครื่อง โดยยังไม่ได้ทดสอบการชำระเงินจริงบนระบบ Production และยังไม่ได้ทดสอบกับผู้ใช้งานจำนวนมาก ดังนั้นผลด้านประสิทธิภาพเมื่อมีผู้ใช้งานพร้อมกันจำนวนมากควรมีการทดสอบเพิ่มเติม

5.3 ข้อจำกัดของการทดลอง

การทดลองครั้งนี้มีข้อจำกัดดังต่อไปนี้:

1. ทดสอบบนเครื่องคอมพิวเตอร์สำหรับพัฒนา ไม่ใช่เซิร์ฟเวอร์จริง

2. ทดสอบข้อมูลผู้ใช้และข้อมูลหนังสือในปริมาณจำกัด
3. ยังไม่ได้ทดสอบการชำระเงินจริงกับบัตรเงินจริง
4. ยังไม่ได้วัดความเร็วเมื่อมีผู้ใช้งานพร้อมกันจำนวนมาก
5. ยังไม่ได้ทดสอบบนอุปกรณ์ Android หลายรุ่น
6. ยังไม่ได้ทดสอบการดาวน์โหลดไฟล์ขนาดใหญ่ในเครือข่ายที่มีความเร็วแตกต่างกัน
7. Lint ยังพบ Warning จากไฟล์ที่สร้างโดย Capacitor/Android จำนวน 16 รายการ แม้จะไม่พบ Error

5.4 ข้อเสนอแนะในการพัฒนาต่อไป

ข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคตมีดังนี้:

1. เพิ่มระบบทดสอบอัตโนมัติสำหรับหน้าเว็บและ API
2. เพิ่มระบบทดสอบการชำระเงินด้วย Stripe Test Mode ให้ครอบคลุมมากขึ้น
3. ทดสอบ Load Test เพื่อวัดประสิทธิภาพเมื่อมีผู้ใช้จำนวนมาก
4. เพิ่มระบบแจ้งเตือนทางอีเมลหลังชำระเงินสำเร็จ
5. เพิ่มระบบรายงานยอดขายและสถิติการดาวน์โหลดสำหรับผู้ดูแลระบบ
6. เพิ่มการรองรับ EPUB อย่างครบถ้วนในส่วนของการอ่านออนไลน์
7. ปรับปรุง Warning จากไฟล์ที่สร้างโดย Capacitor/Android
8. เพิ่มระบบสำรองข้อมูลฐานข้อมูลและไฟล์หนังสือ
9. เพิ่มการตรวจสอบความปลอดภัยของ API อย่างสม่ำเสมอ
10. ทดสอบแอป Android บนอุปกรณ์จริงหลายรุ่นและหลายขนาดหน้าจอ

5.5 บทสรุป

โดยสรุป ระบบ Folio & Co. สามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ในระดับการทดสอบฟังก์ชันหลัก ระบบสามารถแสดงรายการหนังสือ ค้นหาหนังสือ แสดงรายละเอียด จัดการตะกร้า ตรวจสอบสิทธิ์ และป้องกันการดาวน์โหลดโดยไม่ได้รับอนุญาตได้
ผลการ Build ระบบผ่านโดยไม่พบข้อผิดพลาด และเส้นทางหลักของระบบจำนวน 33 routes สามารถสร้างได้สำเร็จ การทดสอบผ่านเว็บเบราว์เซอร์ยังพบว่าหน้าหลัก ระบบค้นหา หน้ารายละเอียดหนังสือ และตะกร้าสินค้าทำงานได้ถูกต้อง
ดังนั้นจึงสรุปได้ว่า ระบบที่พัฒนาขึ้นสามารถนำไปใช้งานเป็นต้นแบบร้านหนังสืออิเล็กทรอนิกส์ได้ แต่อาจต้องมีการทดสอบเพิ่มเติมในด้านการชำระเงินจริง ประสิทธิภาพ ความปลอดภัย และการใช้งานบนอุปกรณ์จำนวนมากก่อนนำไปใช้งานจริงในระดับ Production อย่างเต็มรูปแบบ